



4. Estructuras de control, seleccion y repeticion en Python y C.

Profesor: Juan Moreno García

Índice

1. Objetivos.
2. Bifurcación en Python y C.
3. Bucles en Python y C.
4. Ejemplos.
5. Ejercicios.
6. Conclusiones.

Índice

1. Objetivos.
2. Bifurcación en Python y C.
3. Bucles en Python y C.
4. Ejemplos.
5. Ejercicios.
6. Conclusiones.

1. Objetivos

1. Recordar la **estructura de bifurcación de Python (if)**.
2. Recordar las **estructuras para realizar bucle de Python (for y while)**.
3. Aprender la **estructura de bifurcación de C (if)**.
4. Aprender la **estructura de bucles de C (for y while)**.

2. Bifurcación en Python y C.

Python

```
if condición:
    bloque de sentencias
else:
    bloque de sentencias
```

C

```
if (expresión condicional) {
    sentencia 1;
    sentencia 2;
    ...
    sentencia n;
} else {
    sentencia 1;
    sentencia 2;
    ...
    sentencia m;
}
```

EJEMPLO: Clasifica entrada en bajo, medio o alto según intervalos $[0,10)$, $[10,20)$ o $[21,30)$, en otro caso devuelve “No clasificable”.

Python

```
def selecciona(a):
    if a>=0 and a<10:
        return 'bajo'
    elif a>=10 and a<20:
        return 'medio'
    elif a>=20 and a<=30:
        return 'alto'
    else:
        return 'No clasificable'

v = int( input('Entero entre 0 y 30: ') )
print( selecciona(v) )
```

C

```
String selecciona(int a) {
    if (a>=0 && a<10) {
        return String("bajo");
    } else if (a>=10 && a<20) {
        return String("medio");
    } else if (a>=20 && a<=30) {
        return String("alto");
    } else {
        return String("No clasificable");
    }
}
```

3. Bucles en Python y C.

Python

```
for i in range(1,10):  
    print('i =', i)
```

C

```
for(inicialización; condición; incremento) {  
    sentencia 1;  
  
    ...  
  
    sentencia n;  
}
```

EJEMPLOS

Python

```
def suma_valores_entre(a,b):  
    r = 0;  
    for i in range(a,b+1):  
        r += i  
    return r  
  
a = int( input('Dame un entero: ') )  
b = int( input('Dame un entero: ') )  
print( suma_valores_entre(a,b) )
```

C

```
int suma_valores_entre(int a, int b) {  
    int r, i;  
  
    r = 0;  
    for(i=a; i<b+1; i++) {  
        r += i;  
    }  
    return r;  
}
```

3. Bucles en Python y C.

Cálculo de dosis de paracetamol para un niño.

```
def paracetamol_dosis(kg):  
    mg_kg_hora = 50/24  
    mg_hora = mg_kg_hora*kg  
    ml_hora = mg_hora/40  
    print('Aipiretal 40 para niño de {}kg\n'.format(kg))  
    print('Cada\tmlg\tml')  
    print('----\t--\t--')  
    for horas in range(4,9,2):  
        print('{}h\t{}\t{}\n'.format(horas,  
            round(mg_hora*horas),  
            round(ml_hora*horas)))  
  
paracetamol_dosis(13.5)
```

Aipiretal 40 para niño de 13.5kg

Cada	mg	ml
4h	113	3
6h	169	4
8h	225	6

Kilos = 13.50

Aipiretal 40 para niño de 13.50kgs.

Cada	mg	ml
4	112	3
6	169	4
8	225	6

```
void paracetamol_dosis(float kg) {  
    int horas;  
    float mg_kg_hora, mg_hora, ml_hora;  
  
    mg_kg_hora = (float) 50/24;  
    mg_hora = mg_kg_hora*kg;  
    ml_hora = mg_hora/40;  
    Serial.println( "Aipiretal 40 para niño de " + String(kg) + "kgs." );  
    Serial.println( "Cada\tmlg\tml" );  
    Serial.println( "----\t--\t--" );  
    for(horas=4; horas<9; horas+=2) {  
        Serial.println( String(horas) + "\t" + String(round(mg_hora*horas)) +  
            "\t" + String(round(ml_hora*horas)));  
    }  
}  
  
void setup() {  
    Serial.begin(115200);  
}  
  
void loop() {  
    float kilos;  
    int a, b, c;  
  
    // Recibir entero  
    kilos = leerReal();  
    Serial.println("Kilos = " + String(kilos) );  
  
    // Invocación función  
    paracetamol_dosis(kilos);  
}
```

3. Bucles en Python y C.

Python

```
while condición:  
    bloque sentencias
```

C

```
while(condición) {  
    sentencia 1;  
    sentencia 2;  
    ...  
    sentencia n;  
}
```

EJEMPLO: Función que haga la suma de los valores en el intervalo [a,b] utilizando un bucle while, a y b serán los dos parámetros de la función.

Python

```
def suma_valores_entre(a, b):  
    i = a  
    r = 0  
    while(i<b+1):  
        r += i  
        i += 1  
    return r
```

C

```
int suma_valores_entre(int a, int b) {  
    int r, i;  
  
    i = a;  
    r = 0;  
    while(i<b+1) {  
        r += i;  
        i += 1;  
    }  
    return r;  
}
```


4. Ejemplos.

Implementar una función que dado un número N desde el Monitor serie muestra por pantalla todos los múltiplos de otro número n entre los valores comprendidos en el intervalo $[1, N]$. Ambos números serán los parámetros de la función.

Python

```
def multiplo(e,n):  
    return n%e==0  
  
def multiplos_de(N, n):  
    for e in range(1,N+1):  
        if multiplo(n,e):  
            print(e, 'es multiplo de', n)  
  
N = int( input('Dame el valor máximo del intervalo: ') )  
n = int( input('Dame el valor del que buscar los múltiplos: ') )  
multiplos_de(N, n)
```

4. Ejemplos.

Solución en C

IMPORTANTE:

Nueva función
leerEntero(String s)
que tiene un parámetro con
el texto a utilizar para pedir
la información al usuario



```
int leerEntero(String s) {
    String r;
    Serial.println(s);
    r = leerLinea();
    return r.toInt();
}

String leerLinea() {
    // Espera hasta que haya algo que leer
    while(Serial.available() < 1) {
        delay(1);
    }
    return Serial.readStringUntil('\n');
}
```

```
bool multiplo(int e, int n) {
    return n%e==0;
}

void multiplos_de(int N, int n){
    int e;
    for(e=1;e<N+1;e++) {
        if (multiplo(n,e)) {
            Serial.println( String(e) + " es multiplo de " + String(n) );
        }
    }
    Serial.println();
}

void setup() {
    Serial.begin(115200);
}

void loop() {
    int N, n;

    // Recibir enteros
    N = leerEntero("Dame el valor máximo del intervalo: ");
    n = leerEntero("Dame el valor del que buscar los múltiplos: ");
    Serial.println("Múltiplos de " + String(n) + " entre 1 y " + String(N) );

    // Invocación función
    multiplos_de(N, n);
}
```

5. Ejercicio.

1. Hacer una función que calcule la siguiente expresión:

$$y = \sum_{i=a}^b (i^2) * \left(\frac{i}{(i-1)} \right)$$

2. Implementa la función **ultimo_multiplo_de(N,n)** que dado dos números enteros N y n como argumentos devuelve el múltiplo de n de más valor inferior a N . Si no hay ninguno debe devolver -1. Por ejemplo, si $N=17$ y $n=5$ debe devolver 15.

3. Define una función de nombre *suma_serie* con un argumento n que devuelva el valor de la suma de los primeros n términos de la serie siguiente:

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \dots$$

5. Ejercicio.

4. Un número feo es aquel cuyos únicos divisores primos son 2, 3 o 5. Realizar un programa que lea una secuencia de números hasta que se introduzca el valor -1 y muestre en el Monitor Serie cual de ellos es un número feo. Define una función de nombre *numero_feo* que admite como argumento un número entero y devuelve un *booleano* indicando si el número es feo o no.
5. Define una función de nombre *suma_digitos* que admite como argumento un número entero. Debe devolver un número entero de una sola cifra, resultante de sumar los dígitos del número mientras el resultado tenga más de un dígito.

Por ejemplo, para el número 187 la suma de los dígitos resulta 16, que tiene más de un dígito. Por tanto volvemos a aplicar el mismo procedimiento, que resulta en un 7. Éste ya solo tiene un dígito y por tanto es el resultado final.

5. Ejercicio.

6. Un número es ondulante cuando sus dígitos son alternativamente mayores o menores que los dígitos adyacentes. Por ejemplo, el número 4253612 es ondulante.

Define una función *es_ondulante* que acepta un único argumento String que contiene un número positivo. Debe devolver True si el argumento es ondulante y False en caso contrario.

6. Conclusiones

1. En general, **el funcionamiento de las bifurcaciones y bucles en C es muy similar**, cambia la sintaxis y los operadores a utilizar. En **Python hay más riqueza en el lenguaje**.
2. **Los programas y funciones Python** que no tienen estructuras de datos de alto nivel (listas, tuplas, conjuntos y diccionarios) **son convertibles a código C de una manera relativamente sencilla**.
3. Las **funciones aportadas por el lenguaje pueden funcionar de manera diferente en ambos lenguajes** como se ha comprobado con la función *round()*. Hay que tener cuidado y estudiar cómo trabaja la función a utilizar.
4. Se ha aportado **una nueva versión de la función *leerEntero()*** en este tema.
5. Estas son las diapositivas de clase, para una comprensión en profundidad **se deben estudiar los apuntes que se han colgado en el Campus Virtual**.